TP 4 : Paie, contrats et imprévus

> Vous partez du code de départ fourni en **annexe** (classes `Employe`, `Permanent`, `Commercial` et `ChefProjet`) : recopiez-le avant de commencer, puis enrichissez-le étape par étape.
>
> Comme d'habitude, commencez par **les diagrammes**. Les interfaces portent la mention «interface», les flèches en pointillés se lisent « réalise », et tout ce qui est écrit **en orange** peut lever une exception.

Ce que vous allez pratiquer

- Déclarer une **interface** et la réaliser avec `implements`, y compris plusieurs interfaces à la fois.
- Utiliser une interface fournie par Java, `Comparable<T>`, pour trier un tableau avec `Arrays.sort`.
- Écrire une **constante** et une **méthode par défaut** (`default`) dans une interface.
- Attraper une exception avec `try` / `catch` / `finally` et lire son message avec `getMessage()`.
- Lever une exception avec `throw` et la signaler dans une signature avec `throws`.
- Créer **vos propres exceptions**, vérifiées (`extends Exception`) et non vérifiées (`extends RuntimeException`).

Le déroulé de la séance

Étape	Ce que vous faites	Points
1	Écrire l'interface `Payable` et la classe `Facture`	2
2	Trier les employés grâce à `Comparable`	2
3	Découvrir les exceptions de Java	2
4	Écrire l'exception `MontantInvalideException`	3
5	Écrire l'interface `Augmentable`	3
6	Écrire `Service` et `EquipeCompleteException`	3
7	Le scénario de recrutement	3
8	Les interfaces comme types	2
	Total	**20**

Deux règles pour aujourd'hui

Interdit d'avaler une exception ! Un `catch` vide, ou qui se contente d'un `e.printStackTrace()`, vaut 0 à l'étape. Chaque `catch` affiche un message compréhensible par l'utilisateur.

Comme aux TP précédents, un petit commentaire nomme chaque concept : `// réalisation d'interface`, `// exception vérifiée`, `// levée d'exception`, `// propagation`, `// bloc toujours exécuté`…

CONTEXTE

L'histoire

La société **Atlas Services** gère son personnel avec une petite application Java écrite par l'équipe précédente (le code de l'annexe). Le mois dernier, une saisie de salaire à **-800** est passée inaperçue : le programme l'a discrètement remplacée par 0, et un employé a été enregistré... sans salaire. Personne n'a été prévenu. La direction est formelle : une donnée impossible doit être **refusée et signalée**, jamais corrigée en silence.

Le service RH veut aussi regrouper les employés dans des **services** d'effectif limité, les **classer** du moins payé au mieux payé, et appliquer des **augmentations** encadrées. Enfin, la comptabilité doit pouvoir calculer d'un seul coup tout ce qu'il y a **à payer** : les salaires, mais aussi les factures des fournisseurs. Or une facture n'est pas un employé...

Pour exprimer « peut être payé » ou « peut être augmenté » sans passer par l'héritage, on va écrire des **contrats** : les **interfaces**. Et pour signaler proprement ce qui tourne mal, on va utiliser des **exceptions**, dont deux que l'on fabriquera soi-même.

LES DIAGRAMMES FOURNIS

Les plans de l'application

1. Comment le projet est rangé

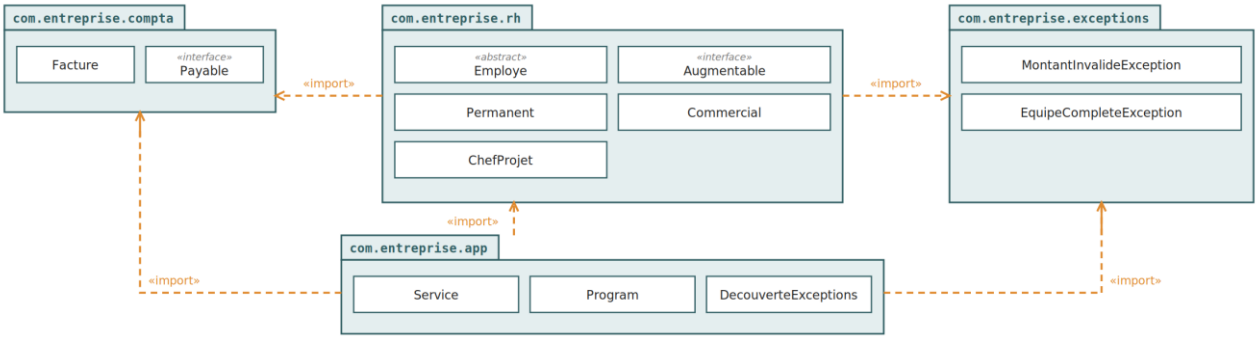
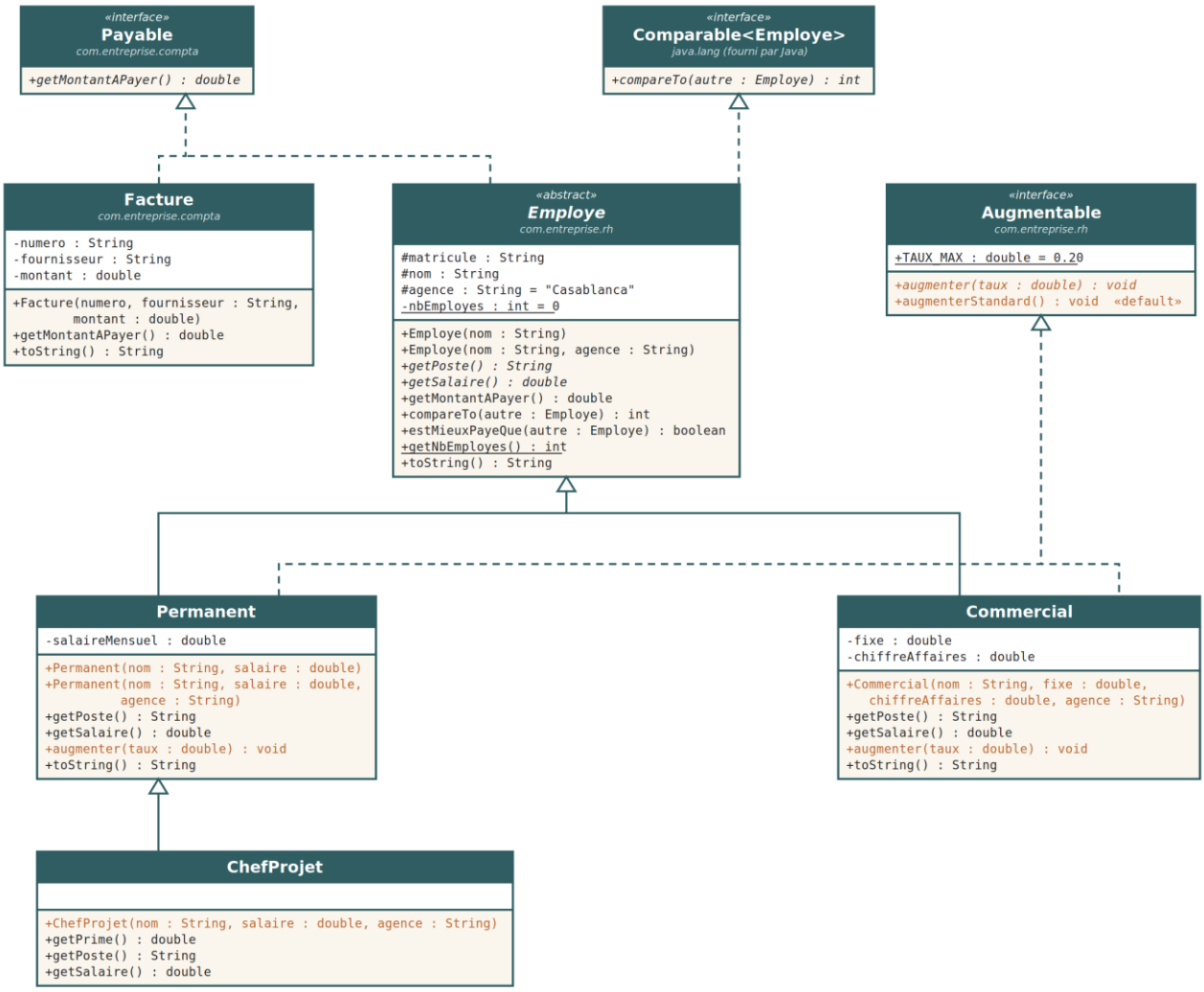


Figure 1 : diagramme de packages. La comptabilité, les ressources humaines et les exceptions ont chacune leur package.

2. Les interfaces et les employés



Trait plein + triangle = « hérite de » (extends) · Pointillés + triangle = « réalise » (implements)
En orange : peut lever une MontantInvalideException · souligné = statique · en italique = abstrait

Figure 2 : diagramme de classes. Une classe n'a qu'une seule mère (extends), mais peut réaliser plusieurs interfaces (implements).

3. La famille des exceptions

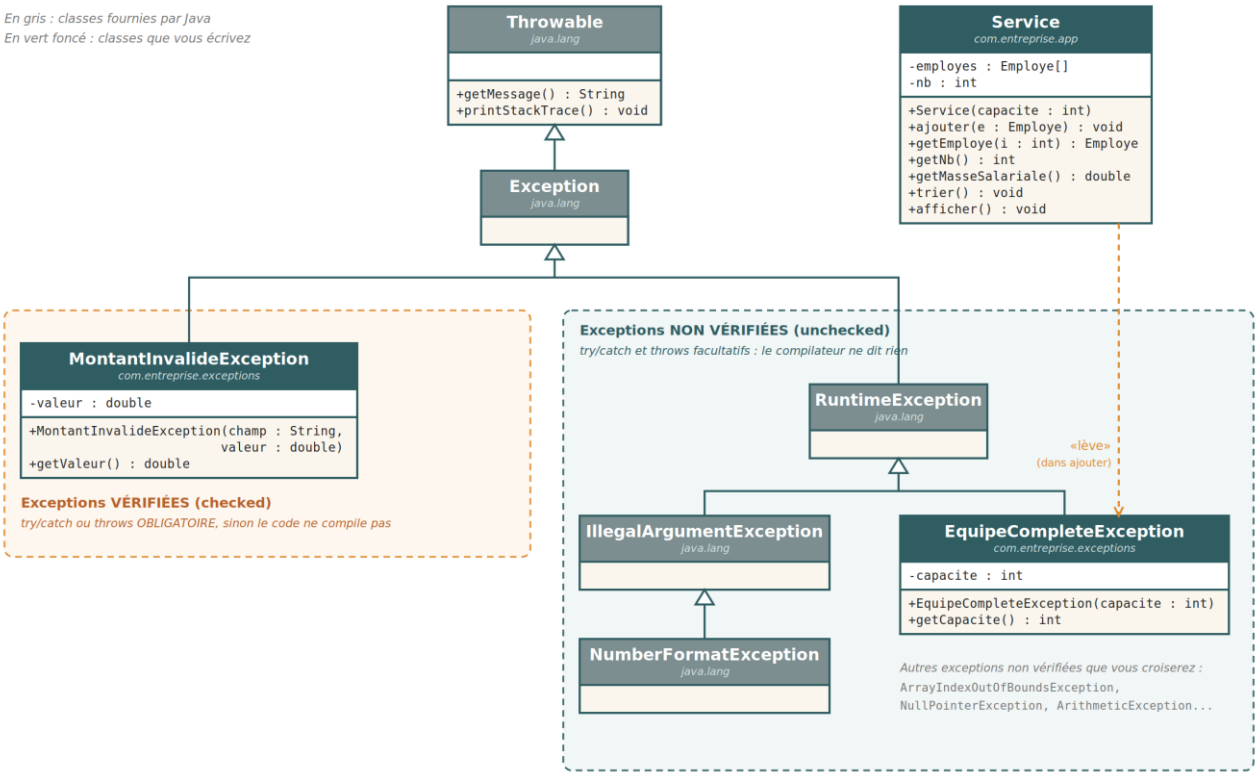


Figure 3 : hiérarchie des exceptions. Tout ce qui descend de RuntimeException est non vérifié ; le reste de la famille Exception est vérifié.

4. Ce qui se passe quand on saisit « -800 »

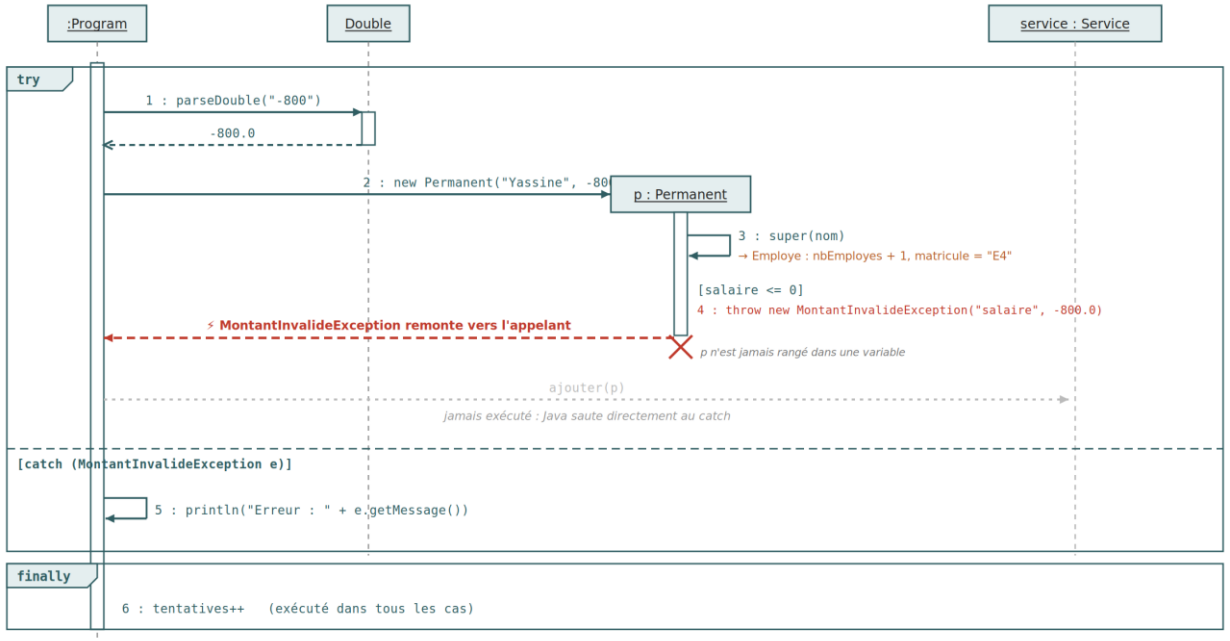


Figure 4 : diagramme de séquences. Dès que throw est exécuté, la fin du try est abandonnée : on saute au catch, puis au finally.

LE TRAVAIL DEMANDÉ

À vous de coder !

1 L'interface Payable et la classe Facture · 2 pts

1. Créez les packages de la **figure 1** et recopiez le code de départ de l'annexe dans `com.entreprise.rh`.
2. Dans `com.entreprise.compta`, créez l'interface `Payable` avec une seule méthode : `double getMontantAPayer();`. Pas de corps, et inutile d'écrire `public` : dans une interface, les méthodes sont automatiquement publiques et abstraites.
3. Toujours dans `com.entreprise.compta`, écrivez la classe `Facture` de la figure 2, qui réalise `Payable` : `getMontantAPayer()` renvoie le montant, et `toString()` renvoie exactement `Facture F-102 (Bureau Plus) : 450.0`.
4. Faites aussi réaliser `Payable` par `Employe` : sa méthode `getMontantAPayer()` (avec `@Override`) renvoie simplement `getSalaire()`.

Coup de pouce

Implémenter une interface, c'est **signer un contrat** : « je promets d'avoir ces méthodes ». Une facture et un employé n'ont aucun lien de parenté, mais ils signent le même contrat. Retirez un instant `getMontantAPayer()` de `Facture` : le compilateur refuse, car la classe a signé sans tenir sa promesse. Remettez-la.

2 Trier les employés grâce à Comparable · 2 pts

1. Faites réaliser à `Employe` une deuxième interface, fournie par Java : `Comparable<Employe>`. La déclaration devient `public abstract class Employe implements Payable, Comparable<Employe>`.
2. Écrivez `compareTo(Employe autre)` : elle renvoie un nombre **négatif** si le salaire de this est plus petit que celui de autre, **0** s'ils sont égaux, un nombre **positif** sinon. Utilisez `Double.compare(...)`.
3. Écrivez `estMieuxPayeQue(Employe autre)` en une seule ligne grâce à `compareTo`.

Coup de pouce

`Comparable` est dans `java.lang` : rien à importer. En échange de cette seule méthode, Java vous offre le tri : `Arrays.sort(tableau)` (dans `java.util`) appelle `compareTo` pour savoir dans quel ordre ranger les employés. C'est tout l'intérêt d'un contrat : vous fournissez `compareTo`, Java fournit le tri.

3 Découvrir les exceptions de Java · 2 pts

Dans `com.entreprise.app`, créez une classe `DecouverteExceptions` avec une méthode `main`. Écrivez-y **quatre petits blocs** `try` / `catch`, un par ligne du tableau. Dans chaque `catch`, affichez le nom de l'exception (`e.getClass().getSimpleName()`) et son message (`e.getMessage()`).

Instruction qui pose problème	Exception à attraper
`double[] primes = new double[3]; primes[5] = 100;`	`ArrayIndexOutOfBoundsException`
`double m = Double.parseDouble("abc");`	`NumberFormatException`
`Facture f = null; f.getMontantAPayer();`	`NullPointerException`
`int partParPersonne = 9000 / 0;`	`ArithmeticException`

1. Ajoutez un bloc `finally` au dernier essai, qui affiche `Fin des essais`. Vérifiez qu'il s'affiche bien alors que l'exception a été levée.
2. Remplacez `9000 / 0` par `9000.0 / 0` : plus d'exception ! Qu'est-ce qui s'affiche, et pourquoi ? Répondez en commentaire.

Petite expérience

Dans un même `try`, placez `catch (Exception e)` **avant** `catch (NumberFormatException e)`. Que vous dit le compilateur ? Déduisez-en la règle sur l'ordre des `catch` et notez-la en commentaire.

4 L'exception MontantInvalideException · 3 pts

1. Dans `com.entreprise.exceptions`, créez `MontantInvalideException`, qui hérite de `Exception` : c'est donc une exception **vérifiée** (figure 3).
2. Ajoutez l'attribut privé `valeur`, son getter, et le constructeur `MontantInvalideException(String champ, double valeur)`. Il appelle `super(...)` avec le message, puis range la valeur. Le message doit avoir exactement la forme `salaire invalide : -800.0`.
3. Dans `Permanent`, supprimez la règle de l'ancienne équipe (« ≤ 0 devient 0 »). À la place, si le salaire est ≤ 0, **levez** l'exception : `throw new MontantInvalideException("salaire", salaire);`.
4. Compilez : erreur ! Ajoutez `throws MontantInvalideException` à la signature des deux constructeurs de `Permanent`. Faites de même dans `Commercial` (champ `"fixe"` s'il est ≤ 0, puis `"chiffreAffaires"` s'il est < 0) et dans `ChefProjet`.

Coup de pouce

`throw` (sans s) **lève** l'exception, ici et maintenant. `throws` (avec s) **prévient**, dans la signature : « attention, je peux lever ça ».

L'exception étant vérifiée, tout code qui appelle `new Permanent(...)` doit soit l'attraper (`try` / `catch`), soit la laisser passer à son tour (`throws`). Si vous aviez déjà écrit un `main` de test, il ne compile plus : mettez-le en commentaire, on écrit le vrai programme à l'étape 7.

5 L'interface Augmentable · 3 pts

Dans `com.entreprise.rh`, créez l'interface `Augmentable` de la figure 2. Elle contient trois éléments :

Élément	Ce qu'il faut écrire
`TAUX_MAX`	Une constante qui vaut 0.20 (20 %). Dans une interface, elle est automatiquement `public static final`
`augmenter(double taux)`	Une méthode abstraite, qui déclare `throws MontantInvalideException`
`augmenterStandard()`	Une méthode **par défaut** (`default`) : elle a un corps, qui appelle simplement `augmenter(0.05)`

1. Faites réaliser `Augmentable` par `Permanent` et par `Commercial`. Si le taux n'est pas dans]0 ; `TAUX_MAX`], levez une `MontantInvalideException("taux", taux)` ; sinon, augmentez le salaire mensuel (pour un commercial : le fixe) de ce pourcentage. Par exemple, 3800 augmenté de 0.05 donne 3990.
2. Vous n'avez rien écrit dans `ChefProjet` : un chef de projet est-il pourtant augmentable ? Expliquez pourquoi en commentaire.

Coup de pouce

Grâce à `default`, toutes les classes qui réalisent `Augmentable` savent `augmenterStandard()` sans l'avoir écrit, du moment qu'elles savent `augmenter`. Remarquez que `augmenterStandard()` doit elle aussi déclarer `throws` : elle appelle une méthode qui peut lever une exception vérifiée, et elle ne l'attrape pas.

6 La classe Service et EquipeCompleteException · 3 pts

1. Dans `com.entreprise.exceptions`, créez `EquipeCompleteException`, qui hérite de `RuntimeException` : elle est **non vérifiée**. Son constructeur reçoit la capacité et produit le message `équipe complète (capacité : 4)`.
2. Dans `com.entreprise.app`, créez la classe `Service` de la figure 3. Elle range des employés dans un tableau de taille fixe et compte les places occupées dans `nb`.

Méthode	Comportement attendu
`Service(int capacite)`	Crée le tableau `employes` de cette taille ; `nb` vaut 0
`ajouter(Employe e)`	Si le tableau est plein, lève une `EquipeCompleteException` ; sinon range e à la case `nb` et augmente `nb`
`getEmploye(int i)`	Si i n'est pas entre 0 et nb - 1, lève une `IllegalArgumentException` avec le message `indice invalide : 7` ; sinon renvoie l'employé
`getNb()`	Renvoie `nb`
`getMasseSalariale()`	Renvoie la somme des salaires des `nb` premiers employés
`trier()`	Trie les `nb` premiers employés avec `Arrays.sort(employes, 0, nb)`
`afficher()`	Affiche chaque employé précédé de son indice : `0 : Permanent[...]`

À vous de réfléchir

`ajouter` n'a pas besoin de `throws` : `EquipeCompleteException` est non vérifiée (vous pouvez l'écrire quand même, pour documenter). Et pour `getEmploye`, inutile de créer une exception : Java fournit déjà `IllegalArgumentException`.

Pourquoi une exception **non vérifiée** pour une équipe complète, mais **vérifiée** pour un montant invalide ? Indice : laquelle des deux erreurs l'appelant peut-il éviter tout seul, en regardant `getNb()` avant d'ajouter ? Répondez en commentaire.

7 Le scénario de recrutement · 3 pts

Dans la classe `Program` (package `com.entreprise.app`), écrivez la méthode `main` :

1. Créez un `Service` de capacité **4**. Dans un `try`, ajoutez-y le commercial `"Karim"` (fixe 3000, chiffre d'affaires 40000, agence `"Rabat"`), puis la cheffe de projet `"Salma"` (salaire 6000, agence `"Casablanca"`). Le `catch` affiche `Erreur :` suivi du message.
2. Les candidats retenus et leurs salaires arrivent sous forme de texte, comme s'ils étaient tapés au clavier : `String[] noms = {"Nadia", "Omar", "Yassine", "Leila", "Hamza"};` et `String[] salaires = {"4500", "abc", "-800", "3800", "5200"};`
3. Pour chaque candidat, dans **un seul** `try` : convertissez son salaire avec `Double.parseDouble`, créez un `Permanent` (agence par défaut), ajoutez-le au service, puis affichez `Embauche de Nadia : 4500.0`.
4. Écrivez **trois** `catch`, un par problème possible (les messages sont dans la console attendue), et un bloc `finally` qui compte les tentatives.
5. Après la boucle, affichez le nombre de tentatives et le nombre d'employés du service.

Coup de pouce

Suivez la figure 4 avec la saisie `"-800"` : dès que `throw` est exécuté, les lignes restantes du `try` sont sautées. Yassine n'est jamais ajouté, et « Embauche » ne s'affiche pas. Le `finally`, lui, s'exécute quoi qu'il arrive.

8 Les interfaces comme types · 2 pts

Toujours dans `main`, à la suite :

1. Triez le service, affichez-le, puis affichez sa masse salariale.
2. Récupérez l'employé le moins payé avec `getEmploye(0)`. Une variable de type `Employe` ne connaît pas `augmenterStandard()` : transtypez-la en `Augmentable` pour l'appeler, puis affichez l'employé. Dans le même `try`, appelez ensuite `augmenter(0.5)` suivi d'un `println("Cette ligne ne s'affiche jamais")`. Le `catch` affiche le message **et** la valeur reçue grâce à `getValeur()`.
3. Appelez `getEmploye(7)` dans un `try` et attrapez l'`IllegalArgumentException`.
4. Créez un tableau `Payable[] paiements` contenant une `new Facture("F-102", "Bureau Plus", 450)` et l'employé d'indice 1 du service. Avec une boucle for-each, affichez chaque élément et calculez le **total à payer**.
5. Terminez en affichant le nombre d'employés créés avec `Employe.getNbEmployes()`.

Le mystère des employés fantômes

Le service compte 4 employés, mais `getNbEmployes()` en annonce **6**. Regardez les matricules affichés : il en manque ! Retrouvez les deux employés « fantômes » et, en vous aidant de la figure 4, expliquez en commentaire pourquoi le compteur a augmenté alors qu'aucun des deux n'est dans le service.

Si tout va bien, votre console affiche ceci :

```
Embauche de Nadia : 4500.0
Saisie ignorée : "abc" n'est pas un montant
Erreur : salaire invalide : -800.0
Embauche de Leila : 3800.0
Erreur : équipe complète (capacité : 4)
Tentatives : 5, employés dans le service : 4
0 : Permanent[matricule=E5, nom=Leila, agence=Casablanca, salaire=3800.0]
1 : Permanent[matricule=E3, nom=Nadia, agence=Casablanca, salaire=4500.0]
2 : Commercial[matricule=E1, nom=Karim, agence=Rabat, fixe=3000.0, chiffreAffaires=40000.0]
3 : ChefProjet[matricule=E2, nom=Salma, agence=Casablanca, salaire=6500.0]
Masse salariale : 19800.0
Après augmentation : Permanent[matricule=E5, nom=Leila, agence=Casablanca, salaire=3990.0]
Erreur : taux invalide : 0.5 [valeur reçue : 0.5]
Erreur : indice invalide : 7
À payer : Facture F-102 (Bureau Plus) : 450.0
À payer : Permanent[matricule=E3, nom=Nadia, agence=Casablanca, salaire=4500.0]
Total à payer : 4950.0
Nombre d'employés créés : 6
```

Pour aller plus loin (si vous avez fini en avance)

Plus de fantômes : écrivez une méthode statique `Permanent.embaucher(String nom, double salaire)` qui vérifie le salaire **avant** d'appeler `new`. Relancez le scénario : les matricules se suivent-ils de nouveau ?

Chaînage d'exceptions : écrivez une méthode statique `lireMontant(String saisie)` qui transforme une `NumberFormatException` en `MontantInvalideException`, en lui passant l'exception d'origine comme **cause**. Ajoutez pour cela un constructeur qui appelle `super(message, cause)`. Que renvoie `getCause()` ?

Faites réaliser `Comparable<Facture>` à `Facture` (comparaison des montants), puis triez un tableau de factures avec `Arrays.sort`.

Avant de rendre votre travail

- ☐ Mes interfaces, classes, attributs et méthodes portent exactement les noms des diagrammes.
- ☐ Aucun `catch` n'est vide : chacun affiche un message utile.
- ☐ `MontantInvalideException` hérite de `Exception`, `EquipeCompleteException` de `RuntimeException`.
- ☐ Chaque méthode qui réalise une méthode d'interface porte `@Override`.
- ☐ Ma console affiche la même chose que l'exemple ci-dessus.
- ☐ J'ai répondu en commentaire aux questions des étapes 3, 5, 6 et 8.

ANNEXE

Le code de départ

Voici le code laissé par l'équipe précédente. Recopiez ces quatre classes dans le package `com.entreprise.rh` (une classe par fichier, chacune commençant par `package com.entreprise.rh;`). Les lignes marquées `// à remplacer` sont celles que vous corrigerez à l'étape 4.

Employe.java

```
public abstract class Employe {
    protected String matricule;
    protected String nom;
    protected String agence;
    private static int nbEmployes = 0;

    public Employe(String nom, String agence) {
        nbEmployes++;
        this.matricule = "E" + nbEmployes;
        this.nom = nom;
        this.agence = agence;
    }

    public Employe(String nom) {
        this(nom, "Casablanca");
    }

    public abstract String getPoste();
    public abstract double getSalaire();

    public static int getNbEmployes() {
        return nbEmployes;
    }

    @Override
    public String toString() {
        return "matricule=" + matricule + ", nom=" + nom + ", agence=" + agence;
    }
}
```

Permanent.java

```
public class Permanent extends Employe {
    private double salaireMensuel;

    public Permanent(String nom, double salaire) {
        super(nom);
        if (salaire <= 0) salaire = 0;   // à remplacer à l'étape 4
        this.salaireMensuel = salaire;
    }

    public Permanent(String nom, double salaire, String agence) {
        super(nom, agence);
        if (salaire <= 0) salaire = 0;   // à remplacer à l'étape 4
        this.salaireMensuel = salaire;
    }

    @Override
    public String getPoste() { return "Permanent"; }

    @Override
    public double getSalaire() { return salaireMensuel; }

    @Override
    public String toString() {
        return getPoste() + "[" + super.toString() + ", salaire=" + getSalaire() + "]";
    }
}
```

Commercial.java

```
public class Commercial extends Employe {
    private double fixe;
    private double chiffreAffaires;

    public Commercial(String nom, double fixe, double chiffreAffaires, String agence) {
        super(nom, agence);
        if (fixe <= 0) fixe = 0;                       // à remplacer à l'étape 4
        if (chiffreAffaires < 0) chiffreAffaires = 0;  // à remplacer à l'étape 4
        this.fixe = fixe;
        this.chiffreAffaires = chiffreAffaires;
    }

    @Override
    public String getPoste() { return "Commercial"; }

    @Override
    public double getSalaire() { return fixe + chiffreAffaires * 5 / 100; }  // fixe + 5 % du CA

    @Override
    public String toString() {
        return getPoste() + "[" + super.toString() + ", fixe=" + fixe
                + ", chiffreAffaires=" + chiffreAffaires + "]";
    }
}
```

ChefProjet.java

```
public class ChefProjet extends Permanent {

    public ChefProjet(String nom, double salaire, String agence) {
        super(nom, salaire, agence);
    }

    public double getPrime() { return 500; }

    @Override
    public String getPoste() { return "ChefProjet"; }

    @Override
    public double getSalaire() { return super.getSalaire() + getPrime(); }
}
```